

Dokumentasi Sistem Watermarking Digital Menggunakan DCT - Johanes Sebastian Siddhadeza (18224062)

Dokumentasi ini menjelaskan alur kerja program untuk menyisipkan data rahasia (*watermark*) ke dalam gambar utama (*host*) menggunakan metode **Discrete Cosine Transform (DCT)**.

DCT dilakukan dengan cara berikut.

1. Membagi gambar menjadi blok-blok kecil berukuran 8×8 piksel
2. Setiap blok diubah dari domain spasial ke domain frekuensi menggunakan transformasi DCT
3. Watermark disisipkan pada koefisien frekuensi menengah
4. Melakukan transformasi balik sehingga gambar dikembalikan ke bentuk semula dengan watermark yang sudah tertanam di dalamnya

Langkah-Langkah pada Kode

1. Import Libraries

```
import cv2
import numpy as np
import os
import matplotlib.pyplot as plt
```

Kegunaan libraries tersebut adalah sebagai berikut.

- cv2 digunakan untuk mengolah gambar, mentransformasi DCT serta sebaliknya, dan memanipulasi gambar
- numpy digunakan untuk operasi matematika untuk memanipulasi matriks
- os digunakan untuk mengelola file direktori dalam sistem
- matplotlib digunakan untuk visualisasi data dalam bentuk gambar dan grafik

2. Memuat dan Menyiapkan Gambar

```
def load_images(host_path, watermark_path):  
    """  
    Loads the host and watermark images.  
    The watermark is converted to grayscale and binarized.  
    """  
    if not os.path.exists(host_path) or not os.path.exists(watermark_path): #check if files exist  
        raise FileNotFoundError(f"Ensure {host_path} and {watermark_path} exist.")  
  
    host = cv2.imread(host_path) # load in color (BGR)  
    if host is None:  
        raise ValueError(f"Could not load host image: {host_path}")  
  
    watermark = cv2.imread(watermark_path, cv2.IMREAD_GRAYSCALE) # load the watermark in grayscale (black or white)  
    if watermark is None:  
        raise ValueError(f"Could not load watermark image: {watermark_path}")  
  
    # Binarize the watermark (0 or 255)  
    _, watermark_bin = cv2.threshold(watermark, 127, 255, cv2.THRESH_BINARY)  
  
    return host, watermark_bin
```

Memastikan gambar mentah dan gambar watermark bisa ditemukan di direktori. Jika file gambar mentah ditemukan, gambar mentah akan di-load dalam RGB. Lalu, gambar watermark akan di-load dalam bentuk grayscale sehingga kemungkinannya hanya bisa hitam dan putih. Setelah diload, watermark akan diubah menjadi bilangan biner.

```
def prepare_watermark(watermark, host_shape, block_size=8):  
    """  
    Resizes the watermark to fit the grid of blocks in the host image.  
    If host is HxW, watermark will be (H/block_size) x (W/block_size).  
    """  
    h, w = host_shape[:2] # height and width of the host image  
    rows = h // block_size # number of blocks vertically  
    cols = w // block_size # number of blocks horizontally  
  
    # Resize using nearest neighbor to preserve binary nature  
    resized_wm = cv2.resize(watermark, (cols, rows), interpolation=cv2.INTER_NEAREST) # resize the watermark to fit the number of blocks in  
    return (resized_wm > 127).astype(np.uint8) # convert to binary (0 and 1)
```

Ukuran watermark kemudian diperkecil secara otomatis agar pas dengan jumlah blok grid yang ada pada gambar utama.

3. Memisahkan Kecerahan dari Warna

```
def embed_watermark(host, watermark, block_size=8, alpha=15):  
    """  
    Embeds binary watermark bits into the DCT coefficients of the host image's Y channel.  
    The embedding modifies the (4,4), a mid-frequency coefficient to balance a good image quality and robustness.  
    """  
    # 1. Color space conversion (BGR -> YCrCb) to process Luminance (Y)  
    ycrCb = cv2.cvtColor(host, cv2.COLOR_BGR2YCrCb) # manipulate the luminance channel for better invisibility  
    y_channel = ycrCb[:, :, 0].astype(np.float32) # convert to float for DCT processing  
  
    wm_h, wm_w = watermark.shape  
  
    # Mid-frequency coefficient choice (4,4) balances robustness and invisibility  
    coeff_pos = (4, 4) # The common choice for mid-frequency embedding, which offers a good balance between invisibility and robustness aga
```

Mengonversi gambar mentah dari RGB menjadi YCrCb agar bisa memanipulasi channel Y. Channel Y yang dimanipulasi karena itu merupakan channel perubahan yang tidak terlalu kelihatan oleh orang lain setelah dimodifikasi. Setelah memanipulasi channel y, gambar

diconvert dalam bentuk float agar bisa diproses DCT. Watermark disisipkan pada koefisien frekuensi menengah karena paling tidak kelihatan perubahannya.

```
# 2. Block-based DCT Embedding
for i in range(wm_h):
    for j in range(wm_w):
        row_start = i * block_size
        col_start = j * block_size

        # Extract 8x8 block
        block = y_channel[row_start:row_start+block_size, col_start:col_start+block_size]

        # Apply Discrete Cosine Transform
        dct_block = cv2.dct(block)

        # Embed bit: Additive modification
        if watermark[i, j] == 1:
            dct_block[coeff_pos] += alpha # Increase the coefficient to embed a '1' bit
        else:
            dct_block[coeff_pos] -= alpha

        # Apply Inverse DCT
        y_channel[row_start:row_start+block_size, col_start:col_start+block_size] = cv2.idct(dct_block)
```

Di kode ini dilakukan transformasi DCT. Setelah ditransformasi, setiap bit pada watermark akan dicek apakah ia white atau black, lalu dilakukan modifikasi untuk setiap warna tersebut untuk menyisipkan watermark. Setelah dilakukan penyisipan, dilakukan invers transformasi DCT untuk mengembalikan frekuensi tadi menjadi gambar piksel normal.

4. Mengekstrak Kembali Watermark

```
def extract_watermark(watermarked_img, host_img, block_size=8):
    # 1. Process the watermarked image (likely the compressed version)
    ycrb_wm = cv2.cvtColor(watermarked_img, cv2.COLOR_BGR2YCrCb)
    y_wm = ycrb_wm[:, :, 0].astype(np.float32)

    # 2. Process the original host for comparison
    ycrb_orig = cv2.cvtColor(host_img, cv2.COLOR_BGR2YCrCb)
    y_orig = ycrb_orig[:, :, 0].astype(np.float32)

    h, w = y_wm.shape
    rows, cols = h // block_size, w // block_size
    extracted_wm = np.zeros((rows, cols), dtype=np.uint8)

    # Synchronized coefficient position (must match embedding)
    coeff_pos = (4, 4)

    # 3. Block-based Extraction
    for i in range(rows):
        for j in range(cols):
            row_s, col_s = i * block_size, j * block_size

            # Extract and transform blocks from both images
            block_wm = cv2.dct(y_wm[row_s:row_s+block_size, col_s:col_s+block_size])
            block_orig = cv2.dct(y_orig[row_s:row_s+block_size, col_s:col_s+block_size])

            # Logic: If the watermarked coefficient is greater than the original,
            # it indicates a bit '1' was embedded (assuming alpha > 0).
            if block_wm[coeff_pos] > block_orig[coeff_pos]:
                extracted_wm[i, j] = 255 # White pixel
            else:
                extracted_wm[i, j] = 0 # Black pixel

    return extracted_wm
```

Membaca dan menampilkan kembali logo/pesan rahasia dari gambar yang dicurigai untuk membuktikan kepemilikan. Program memotong gambar ber-watermark dan gambar asli menjadi blok 8x8, lalu mengecek frekuensi (4,4)-nya. Jika nilai frekuensi gambar ber-watermark lebih besar daripada gambar asli, berarti itu adalah bit 1 (Putih). Jika lebih kecil, berarti bit 0 (Hitam).

5. Evaluasi Kualitas & Uji Ketahanan

Evaluasi dilakukan dalam 2 tahap:

- Uji Kualitas (Fidelity): Menggunakan rumus PSNR dan SSIM untuk memastikan gambar yang sudah disisipi watermark secara visual tetap terlihat sama persis dengan aslinya.
- Uji Ketahanan (Robustness Test): Gambar sengaja "disiksa" dengan kompresi JPEG (kualitasnya diturunkan dari 100 hingga 10). Program lalu mengukur Bit Error Rate (BER) untuk melihat di kualitas JPEG berapa watermark tersebut akhirnya rusak dan tidak bisa dibaca lagi.

Evaluasi Kinerja Watermark

Melihat Quality Factor dari rentang 10 sampai 100 dengan interval 10.

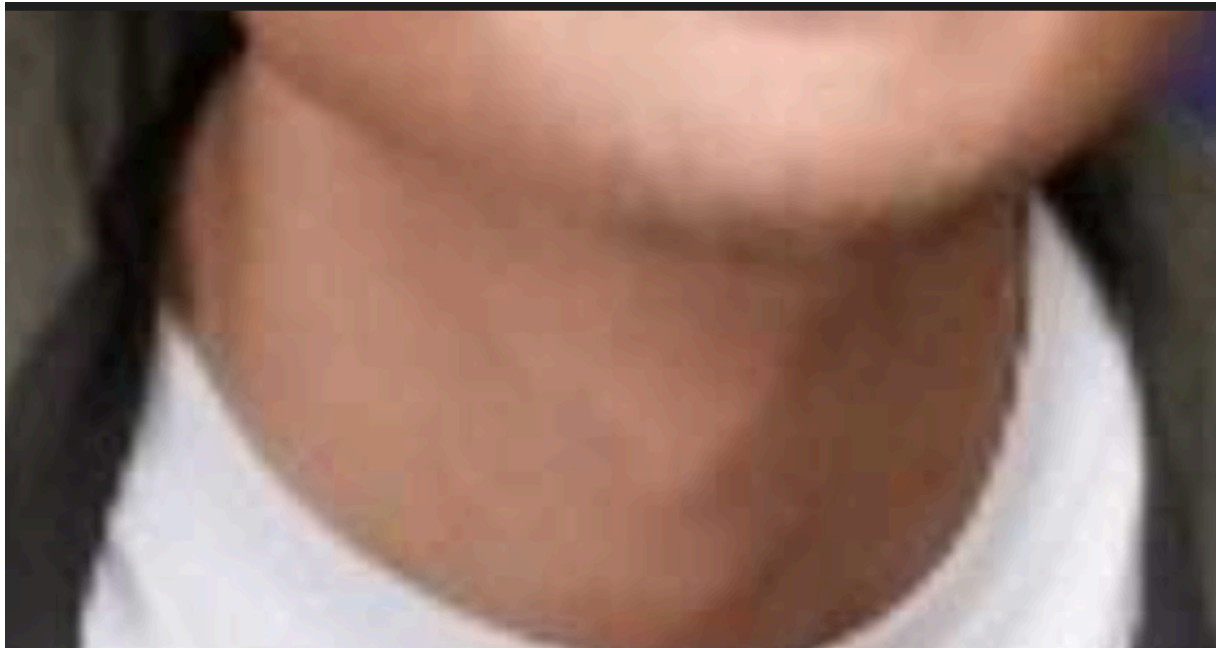
100	0.0000	RECOVERABLE
90	0.0000	RECOVERABLE
80	0.0001	RECOVERABLE
70	0.0010	RECOVERABLE
60	0.0046	RECOVERABLE
50	0.6835	!!! FAILED !!!
40	0.6915	FAILED
30	0.6733	FAILED
20	0.6893	FAILED
10	0.6943	FAILED
[*] Saved successful sample (QF 60) to: sample_success_qf60.jpg		

Berdasarkan hasil evaluasi, gambar saya bisa diekstrak watermarknya pada saat $QF \geq 60$, namun tidak bisa kalau $QF < 60$. Karena itu, ketika watermarknya diekstrak ketika $QF < 60$, maka gambar saya akan jadi buram dan tidak sebagus aslinya. Berikut di bawah merupakan perbandingannya.

QF 60 (berhasil)



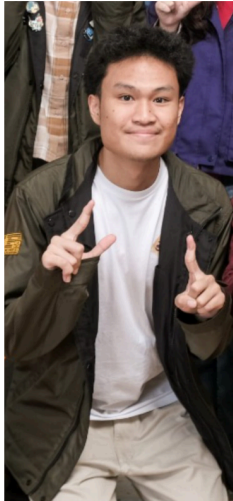
QF 50 (gagal)



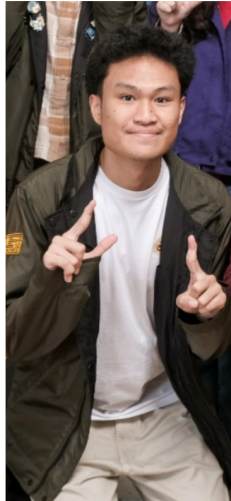
Laporan Perubahan Foto Setelah Diberikan Watermark

IMAGE FIDELITY ANALYSIS: Does the watermark ruin the photo?

1. Original Photo



2. Watermarked Photo
PSNR: 37.18 dB



3. Where pixels changed?
(Difference Map x10 for visibility)

